

Date:
Student Name:
Student ID:

AI Workshop #6

KNN for Building Recommendation Systems

6.1. Introduction

In this workshop, we will learn how to build a recommendation system that will recommend movies that people might like to watch.

We will learn about the **K-nearest neighbors classifier (KNN)** and see how to implement it. We use these concepts to discuss collaborative filtering and then use it to build a recommender system.

By the end of this workshop, you will have a better understanding of these topics:

- Extracting the nearest neighbors
- Building a K-Nearest Neighbors classifier
- Computing similarity scores
- Finding similar users using collaborative filtering
- Mini-project: Building a movie recommendation system

6.2 Extracting the nearest neighbors (KNN)

Recommender systems employ the concept of nearest neighbors to find good recommendations.

The name **nearest neighbors** refers to the process of finding the closest data points to the input point from the given dataset.

This is frequently used to build classification systems that classify a data point based on the proximity of the input data point to various classes.

Let's see how to find the nearest neighbors for a given data point.

1. First, create a new Python file and import the following packages:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
from sklearn.neighbors import NearestNeighbors
```

2. Define sample 2D data points (as example):

```
# Input data
X = np.array([[2.1, 1.3], [1.3, 3.2], [2.9, 2.5], [2.7, 5.4], [3.8, 0.9],
              [7.3, 2.1], [4.2, 6.5], [3.8, 3.7], [2.5, 4.1], [3.4, 1.9],
              [5.7, 3.5], [6.1, 4.3], [5.1, 2.2], [6.2, 1.1]])
```

3. Define the number of nearest neighbors you want to extract (k=5 in our case):

```
# Number of nearest neighbors
k = 5
```

4. Define a test data point that will be used to extract the K-nearest neighbors:

```
# Test datapoint
test_datapoint = [4.3, 2.7]
```

5. Plot the input data using circular black markers:

```
# Plot input data
plt.figure()
plt.title('Input data')
plt.scatter(X[:,0], X[:,1], marker='o', s=75, color='black')
```

6. Create and train a K-nearest neighbors model using the input data. Use this model to extract the nearest neighbors to the test data point:

```
# Build K Nearest Neighbors model
knn_model = NearestNeighbors(n_neighbors=k, algorithm='ball_tree').fit(X)
distances, indices = knn_model.kneighbors([test_datapoint])
```

7. Print the nearest neighbors extracted from the model:

```
# Print the 'k' nearest neighbors
print("\nK Nearest Neighbors:")
for rank, index in enumerate(indices[0][:k], start=1):
    print(str(rank) + " ==>", X[index])
```

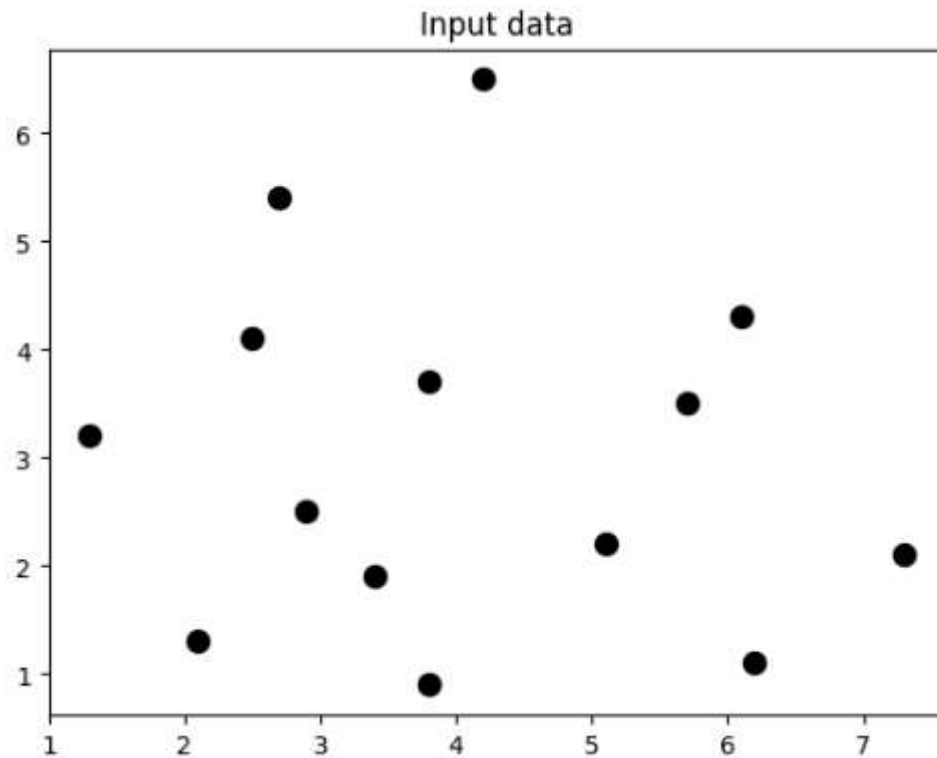
8. Visualize the nearest neighbors:

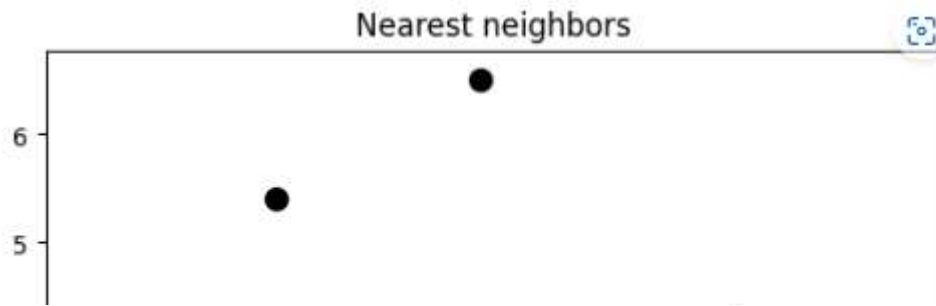
```
# Visualize the nearest neighbors along with the test datapoint
plt.figure()
plt.title('Nearest neighbors')
```

```
plt.scatter(X[:, 0], X[:, 1], marker='o', s=75, color='k')
plt.scatter(X[indices][0][:][:, 0], X[indices][0][:][:, 1],
            marker='o', s=250, color='k', facecolors='none')
plt.scatter(test_datapoint[0], test_datapoint[1],
            marker='x', s=75, color='k')

plt.show()
```

9. If everything goes well. You should see the following 3 figures.





1. The first screenshot represents the input data.
2. The second screenshot represents the five nearest neighbors. The test data point is shown using a cross and the nearest neighbor points have been circled.
3. The third figure shows the five points that are closest to the test data point.



Workshop 6.1 : K Nearest Neighbors (KNN) with your own implementation

1. Combine all the above code and named it as **knn_system.py**.
2. If everything is okay, you should see the above figures: input data points, marked KNN = 5 data points with the test point and the 5 K-NN points details.
3. Save and submit the source code and these figures.
4. Actually the KNN is just the combination of distance calculation + data point sorting with distance. To implement of your own KNN, try to write the KNN algorithm.
5. Implement the `knn(k, in_data)` function with Python, where `k` is the KNN number, `in_data` and `out_data` are the lists of input data points and output data points respectively.
6. Run and submit your **knn_own.py** program and compare it with the Python NearestNeighbors package in terms of running time. Which one is faster? Why?



Now that we have learned how to construct and run a K-nearest neighbors model. In the next section, we will build on that knowledge and use it to build a **K-nearest neighbors classifier**.

6.3 Building a K-nearest neighbors classifier

A K-nearest neighbors classifier is a classification model that uses the **K-nearest neighbors** algorithm to classify a given data point.

The algorithm finds the K closest data points in the training dataset to identify the category of the input data point. It will then assign a class to this data point based on a majority vote.

From the list of those K data points, we look at the corresponding classes and pick the one with the highest number of votes. The value of K will depend on the problem at hand.

Let's see how to build a classifier using this model.

1. Create a new Python file and import the following packages:

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.cm as cm
from sklearn import neighbors, datasets
```

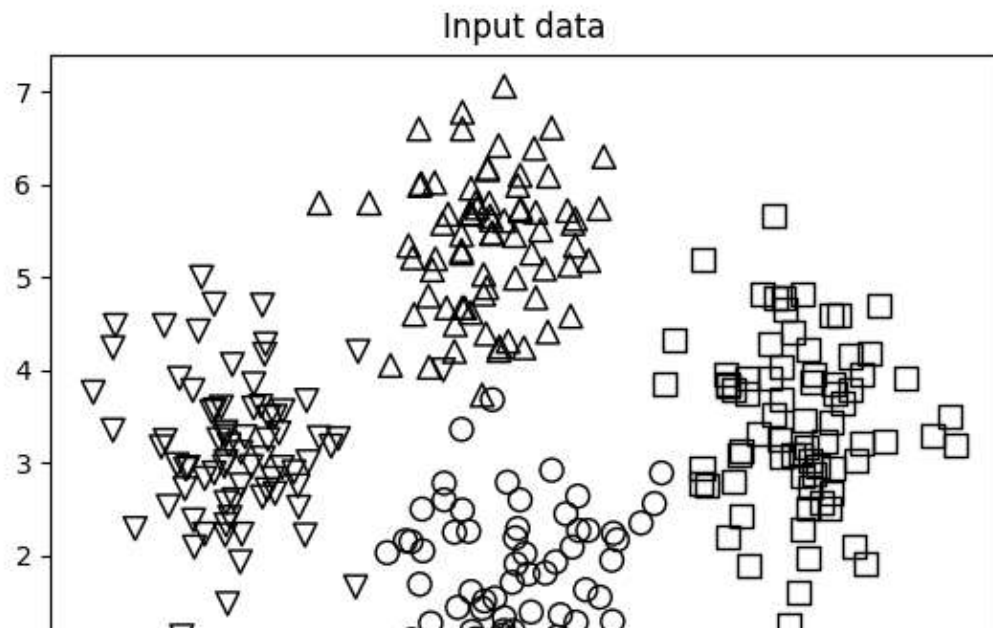
2. Load the input data from data.txt. Each line contains comma-separated values and the data contains four classes:

```
# Load input data
input_file = 'data.txt'
data = np.loadtxt(input_file, delimiter=',')
X, y = data[:, :-1], data[:, -1].astype(int)
```

3. Visualize the input data using four different marker shapes. We need to map the labels to corresponding markers, which is where the mapper variable comes into the picture:

```
# Plot input data
plt.figure()
plt.title('Input data')
marker_shapes = 'v^os'
mapper = [marker_shapes[i] for i in y]
for i in range(X.shape[0]):
    plt.scatter(X[i, 0], X[i, 1], marker=mapper[i],
                s=75, edgecolors='black', facecolors='none')
```

You should see this figure:



4. Define the number of nearest neighbors to be used:

```
# Number of nearest neighbors  
num_neighbors = 12
```

5. Define the step size of the grid that will be used to visualize the boundaries of the classifier model:

```
# Step size of the visualization grid  
step_size = 0.01
```

6. Create the K-nearest neighbors classifier model:

```
# Create a K Nearest Neighbours classifier model  
classifier = neighbors.KNeighborsClassifier(num_neighbors, weights='distance')
```

7. Train the model using the training data:

```
# Train the K Nearest Neighbours model  
classifier.fit(X, y)
```

8. Create the mesh grid of values that will be used to visualize the grid:

```
# Create the mesh to plot the boundaries  
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1  
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1  
x_values, y_values = np.meshgrid(np.arange(x_min, x_max, step_size),
```

```
np.arange(y_min, y_max, step_size))
```

9. Evaluate the classifier on all the points on the grid to create a visualization of the boundaries:

```
# Evaluate the classifier on all the points on the grid
output = classifier.predict(np.c_[x_values.ravel(), y_values.ravel()])
```

10. Create a color mesh to visualize the output:

```
# Visualize the predicted output
output = output.reshape(x_values.shape)
plt.figure()
plt.pcolormesh(x_values, y_values, output, cmap=cm.Paired)
```

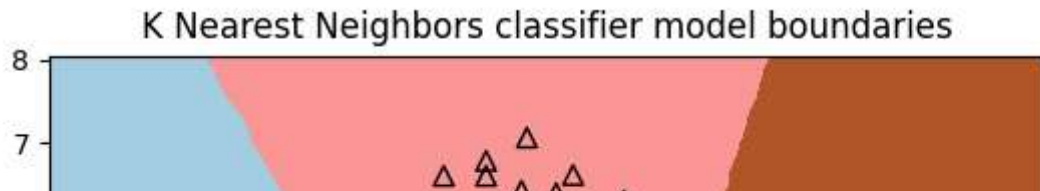
11. Overlay the training data on top of this color mesh to visualize the data relative to the boundaries:

```
# Overlay the training points on the map
for i in range(X.shape[0]):
    plt.scatter(X[i, 0], X[i, 1], marker=mapper[i],
                s=50, edgecolors='black', facecolors='none')
```

12. Set the X and Y limits along with the title:

```
plt.xlim(x_values.min(), x_values.max())
plt.ylim(y_values.min(), y_values.max())
plt.title('K Nearest Neighbors classifier model boundaries')
```

You should see this figure that represents the classifier boundaries:

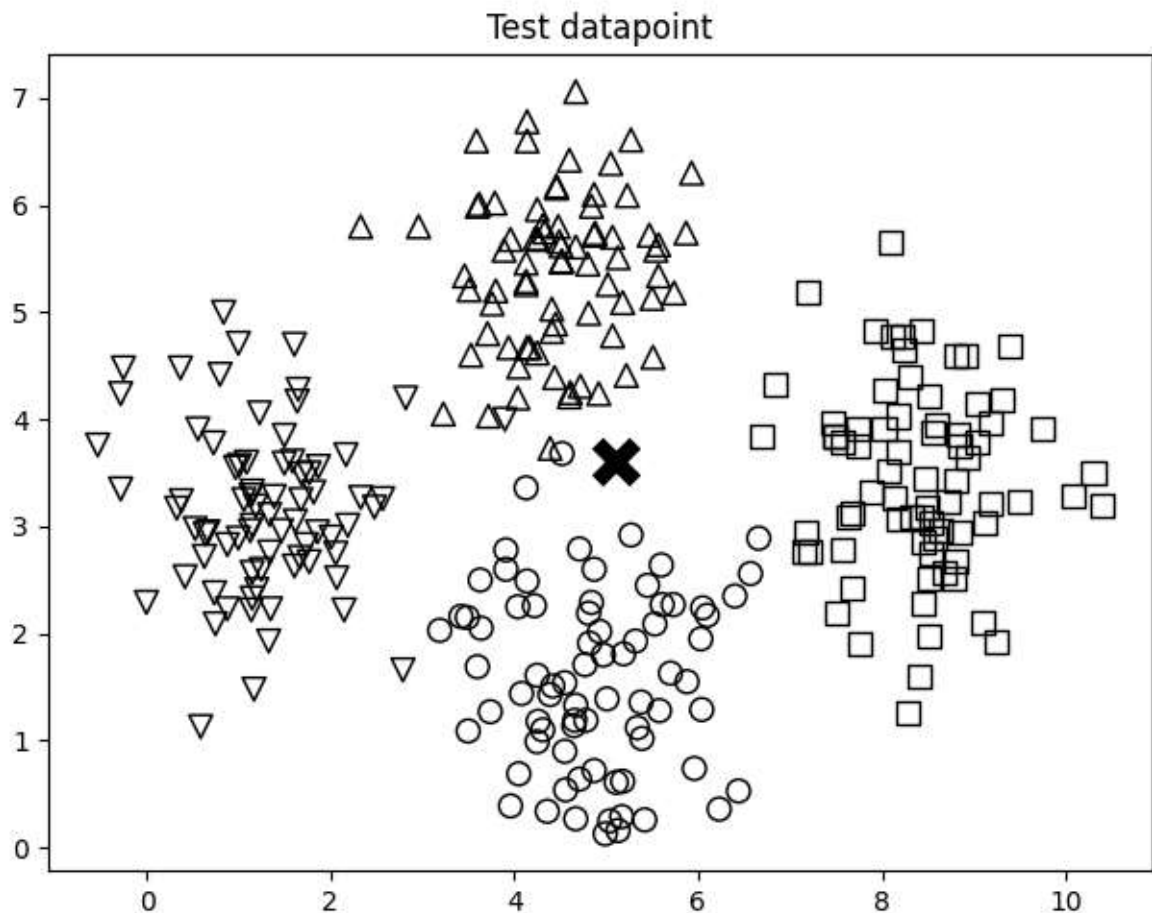


13. Define a test data point to see how the classifier performs. Create a figure with training data points and a test data point to see where it lies:

```
# Test input datapoint
test_datapoint = [5.1, 3.6]
plt.figure()
plt.title('Test datapoint')
for i in range(X.shape[0]):
    plt.scatter(X[i, 0], X[i, 1], marker=mapper[i],
                s=75, edgecolors='black', facecolors='none')

plt.scatter(test_datapoint[0], test_datapoint[1], marker='x',
            linewidth=6, s=200, facecolors='black')
```

You should see this figure where X is the test point:



14. Extract the K-nearest neighbors to the test data point, based on the classifier model:

```
# Extract the K nearest neighbors
_, indices = classifier.kneighbors([test_datapoint])
indices = indices.astype(int)[0]
```

15. Plot the K-nearest neighbors obtained in the previous step:

```
# Plot k nearest neighbors
plt.figure()
plt.title('K Nearest Neighbors')

for i in indices:
    plt.scatter(X[i, 0], X[i, 1], marker=mapper[y[i]],
                linewidth=3, s=100, facecolors='black')
```

16. Overlay the test data point:

```
plt.scatter(test_datapoint[0], test_datapoint[1], marker='x',
            linewidth=6, s=200, facecolors='black')
```

17. Overlay the input data:

```
for i in range(X.shape[0]):
    plt.scatter(X[i, 0], X[i, 1], marker=mapper[i],
                s=75, edgecolors='black', facecolors='none')
```

You should see this figure which shows the 12 nearest neighbors to the test data point:



18. Print the predicted output:

```
print("Predicted output:", classifier.predict([test_datapoint])[0])

plt.show()
```

You will see the following output, indicating that the model is predicting that the test data point belongs to class 1:

Predicted output: 1



Workshop 6.2 : KNN Classifier with your own implementation

1. Combine all the above code and named it as **knn_classifier_system.py**.
2. If everything is okay, you should see the above 4 figures and the predicted results.
3. Save and submit the source code and these figures.
4. Actually the KNN Classifier is a simple extension of the KNN algorithm with the inclusion of the class category. Try your own by writing the KNN Classifier algorithm.
5. Implement the **knn_classifier** function with Python,
6. Run and submit your **knn_classifier_own.py** program and compare it with the Python package in terms of running time. Which one is faster? Why?

6.4 Computing similarity scores

To build a recommendation system, it is important to understand how to compare various objects in the dataset.

If the dataset consists of people and their various movie preferences, then in order to make a recommendation we need to understand how to compare any two people with one another.

This is where the similarity score is important. The similarity score gives an idea of how similar two data points are.

There are two scores that are used frequently in this domain – the **Euclidean score** and the **Pearson score**. The Euclidean score uses the Euclidean distance between two data points to compute the score.

The value of the **Euclidean distance** can be unbounded. Hence, we take this value and convert it in a way that the Euclidean score ranges from 0 to 1. If the Euclidean distance between two objects is large, then the Euclidean score should be low because a low score indicates that the objects are not similar. Hence Euclidean distance is inversely proportional to Euclidean score.

The Euclidean distance of two point p,q is given by:

$$d(\mathbf{p}, \mathbf{q}) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2 + \cdots + (p_n - q_n)^2} = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}.$$

The **Pearson score** is a measure of correlation between two data points. It uses the covariance between the two data points along with their individual standard deviations to compute the score. The score can range from -1 to +1. A score of +1 indicates that the data points are similar, and a score of -1 indicates that they are dissimilar. A score of 0 indicates that there is no correlation between them.

The Pearson correlation of two point X,Y is given by:

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}$$

Let's see how to compute these scores.

1. Create a new Python file and import the following packages:

```
import argparse
import json
import numpy as np
```

2. Build an argument parser to process the input arguments. It will accept two users and the type of score that it needs to use to compute the similarity score:

```
def build_arg_parser():
    parser = argparse.ArgumentParser(description='Compute similarity score')
    parser.add_argument('--user1', dest='user1', required=True,
                        help='First user')
    parser.add_argument('--user2', dest='user2', required=True,
                        help='Second user')
    parser.add_argument("--score-type", dest="score_type", required=True,
                        choices=['Euclidean', 'Pearson'], help='Similarity metric to be used')
```

```
return parser
```

3. Define a function to compute the Euclidean score between the input users. If the users are not in the dataset, the code will raise an error:

```
# Compute the Euclidean distance score between user1 and user2
def euclidean_score(dataset, user1, user2):
    if user1 not in dataset:
        raise TypeError('Cannot find ' + user1 + ' in the dataset')

    if user2 not in dataset:
        raise TypeError('Cannot find ' + user2 + ' in the dataset')
```

4. Define a variable to track the movies that have been rated by both users:

```
# Movies rated by both user1 and user2
common_movies = {}
```

5. Extract the movies rated by both users:

```
for item in dataset[user1]:
    if item in dataset[user2]:
        common_movies[item] = 1
```

6. If there are no common movies, then the similarity score cannot be computed:

```
# If there are no common movies between the users,
# then the score is 0
if len(common_movies) == 0:
    return 0
```

7. Compute the squared differences between the ratings and use it to compute the **Euclidean score**:

```
squared_diff = []

for item in dataset[user1]:
    if item in dataset[user2]:
        squared_diff.append(np.square(dataset[user1][item] - dataset[user2]
[item]))

return 1 / (1 + np.sqrt(np.sum(squared_diff)))
```

8. Define a function to compute the **Pearson score** between the users in the given dataset. If the users are not found in the dataset, it raises an error:

```
# Compute the Pearson correlation score between user1 and user2
def pearson_score(dataset, user1, user2):
    if user1 not in dataset:
        raise TypeError('Cannot find ' + user1 + ' in the dataset')

    if user2 not in dataset:
        raise TypeError('Cannot find ' + user2 + ' in the dataset')
```

9. Define a variable to track the movies that have been rated by both users:

```
# Movies rated by both user1 and user2
common_movies = {}
```

10. Extract the movies rated by both users:

```
for item in dataset[user1]:
    if item in dataset[user2]:
        common_movies[item] = 1
```

11. If there are no common movies, then we cannot compute the similarity score:

```
num_ratings = len(common_movies)

# If there are no common movies between user1 and user2, then the score is 0
if num_ratings == 0:
    return 0
```

12. Calculate the sum of ratings of all the movies that have been rated by both users:

```
# Calculate the sum of ratings of all the common movies
user1_sum = np.sum([dataset[user1][item] for item in common_movies])
user2_sum = np.sum([dataset[user2][item] for item in common_movies])
```

13. Calculate the sum of squares of the ratings all the movies that have been rated by both the users:

```
# Calculate the sum of squares of ratings of all the common movies
user1_squared_sum = np.sum([np.square(dataset[user1][item]) for item in
common_movies])
user2_squared_sum = np.sum([np.square(dataset[user2][item]) for item in
common_movies])
```

14. Calculate the sum of products of the ratings of all the movies rated by both the input users:

```
# Calculate the sum of products of the ratings of the common movies
```

```
sum_of_products = np.sum([dataset[user1][item] * dataset[user2][item] for item in
common_movies])
```

15. Calculate the various parameters required to compute the Pearson score using the preceding computations:

```
# Calculate the Pearson correlation score
Sxy = sum_of_products - (user1_sum * user2_sum / num_ratings)
Sxx = user1_squared_sum - np.square(user1_sum) / num_ratings
Syy = user2_squared_sum - np.square(user2_sum) / num_ratings
```

16. If there is no deviation, then the score is 0:

```
if Sxx * Syy == 0:
    return 0
```

17. Return the Pearson score:

```
return Sxy / np.sqrt(Sxx * Syy)
```

18. Define the main function and parse the input arguments:

```
if __name__ == '__main__':
    args = build_arg_parser().parse_args()
    user1 = args.user1
    user2 = args.user2
    score_type = args.score_type
```

19. Load the ratings from the file ratings.json into a dictionary:

```
ratings_file = 'ratings.json'

with open(ratings_file, 'r') as f:
    data = json.loads(f.read())
```

20. Compute the similarity score based on the input arguments:

```
if score_type == 'Euclidean':
    print("\nEuclidean score:")
    print(euclidean_score(data, user1, user2))
else:
    print("\nPearson score:")
    print(pearson_score(data, user1, user2))
```



Workshop 6.3 : Compute Similarity with Euclidean and Pearson Scores

1. Combine all the above code and name it as **compute_scores.py**
2. Use "David Smith" and "Bill Duffy" as example.
3. Compute the Euclidean score by using:

```
python compute_scores.py --user1 "David Smith" --user2 "Bill Duffy" --score-type Euclidean
```

4. What is the Euclidean score?
5. Compute the Pearson Score by using:

```
python3 compute_scores.py --user1 "David Smith" --user2 "Bill Duffy" --score-type Pearson
```

6. What is the Pearson score?
7. Save and submit the program code and score answers.
8. Find two other actors and compare their Euclidean and Pearson scores.
9. In a two-dimensional space, the **Manhattan distance** between two points (x1, y1) and (x2, y2) would be calculated as: distance = |x2 - x1| + |y2 - y1|. Or in general:

$$\text{Manhattan Distance } (p, q) = \sum_{i=1}^n |p_i - q_i|$$

10. For your program, add the **Manhattan distance** implementation as additional score calculation and named as "**Manhattan**".
11. Using "David Smith" and "Bill Duffy" as example, compare their Euclidean, Pearson and Manhattan scores. Which one you think is better representation of similarity? Why?



In this section, we learned how to compute **similarity scores** and learned why that an important component in the construction of a recommender system. In the next section, we will learn how to identify users with similar preferences by using **collaborative filtering**.

6.5 Finding similar users using collaborative filtering

Collaborative filtering refers to the process of identifying patterns among the objects in a dataset in order to decide about a new object.

In the context of recommendation engines, collaborative filtering is used to provide recommendations by looking at similar users in the dataset.



By collecting the preferences of different users in the dataset, we collaborate that information to filter the users. Hence the name **collaborative filtering**.

The assumption here is that if two people have similar ratings for a set of movies, then their choices for a set of new unknown movies would be similar too.

By identifying patterns in those common movies, predictions can be made about new movies.

In the previous section, we learned how to compare different users in the dataset.

The scoring techniques discussed will now be used to find similar users in the dataset.

Collaborative filtering algorithms can be parallelized and be implemented in big data systems such as **AWS EMR** and **Apache Spark**, enabling the processing of hundreds of terabytes worth of data.

These methods can be used for various verticals like finance, online shopping, marketing, customer studies, among others.

Let's get started and build our collaborative filtering system.

1. Create a new Python file and import the following packages:

```
import argparse
import json
import numpy as np

from compute_scores import pearson_score
```

2. Define a function to parse the input arguments. The input argument is the name of the user:

```
def build_arg_parser():
    parser = argparse.ArgumentParser(description='Find users who are similar to the input user')
    parser.add_argument('--user', dest='user', required=True,
                        help='Input user')
```

```
return parser
```

3. Define a function to find the users in the dataset that are similar to the given user. If the user is not in the dataset, raise an error:

```
# Finds users in the dataset that are similar to the input user
def find_similar_users(dataset, user, num_users):
    if user not in dataset:
        raise TypeError('Cannot find ' + user + ' in the dataset')
```

4. The function to compute the Pearson score has been imported. Let's use that function to compute the Pearson score between the input user and all the other users in the dataset:

```
# Compute Pearson score between input user
# and all the users in the dataset
scores = np.array([[x, pearson_score(dataset, user,
                                     x)] for x in dataset if x != user])
```

5. Sort the scores in descending order:

```
# Sort the scores in decreasing order
scores_sorted = np.argsort(scores[:, 1])[:, -1]
```

6. Extract the top num_users number of users as specified by the input argument and return the array:

```
# Extract the top 'num_users' scores
top_users = scores_sorted[:num_users]

return scores[top_users]
```

7. Define the main function and parse the input arguments to extract the name of the user:

```
if __name__ == '__main__':
    args = build_arg_parser().parse_args()
    user = args.user
```

8. Load the data from the movie ratings file ratings.json. This file contains the names of the people and their ratings for various movies:

```
ratings_file = 'ratings.json'

with open(ratings_file, 'r') as f:
```

```
data = json.loads(f.read())
```

9. Find the top three users who are like the user specified by the input argument. You can change it to any number of users depending on your choice. Print the output along with the scores:

```
print('\nUsers similar to ' + user + ':\n')
similar_users = find_similar_users(data, user, 3)
print('User\t\t\tSimilarity score')
print('-'*41)
for item in similar_users:
    print(item[0], '\t\t', round(float(item[1]), 2))
```



Workshop 6.4 : Collaborative Filtering

1. Combine all the above code and name it as **collaborative_filtering.py**
2. Use "Bill Duffy" as example to find the the users who are like Bill Duffy by using the following command:

python collaborative_filtering.py --user "Bill Duffy"

3. Save and submit the program code and output results.
4. Check the results by using "Samuel Miller" and "David Smith". What is your findings?
5. What are the major pros and cons of Collaborative Filtering?



In this section, we learned how we can find users in a dataset that are like each other as well as being able to assign a score to determine how similar a user is to another. In the next section, we will put it all together and build our recommendation system.



6.6 Mini-project: Building a movie recommendation system

So far, we have laid the foundation to build our recommendation system by learning about:

- Extracting the nearest neighbors
- Building a K-nearest neighbors classifier
- Computing similarity scores
- Finding similar users using collaborative filtering

Now that all the building blocks in place, it's time to build a movie recommendation system. We learned all the underlying concepts that are needed to build a recommendation system.

In this mini-project, we will build a **movie recommendation system** based on the data provided in the file ratings.json.

This file contains a set of people and their ratings for various movies.

To find movie recommendations for a given user, we need to find similar users in the dataset and then come up with recommendations for this person.

Here are the basic steps:

1. Create a new Python file and import the following packages.
2. Define a function to parse the input arguments. The input argument is the name of the user.
3. Define a function **get_recommendations(dataset, input_user)** to get the movie recommendations for a given user. If the user doesn't exist in the dataset, the code will raise an error.
4. Define the variables **overall_scores** and **similarity_scores** to track the scores.
5. Compute a similarity score (e.g. Pearson score) between the input user and all the other users in the dataset.
6. If the similarity score is less than 0, you can continue with the next user in the dataset.
7. Extract a list of movies that have been rated by the current user but haven't been rated by the input user.
8. For each item in the filtered list, keep a track of the weighted rating based on the similarity score. Also keep a track of the similarity scores.

9. If there are no such movies, then we cannot recommend anything.
10. Normalize the scores based on the weighted scores.
11. Sort the scores and extract the movie recommendations.
12. Define the main function and parse the input arguments to extract the name of the input user.
13. Load the movie ratings data from the file ratings.json.
14. Extract the movie recommendations and print the output.
15. Run the program by using the following command with "Chris Duncan" as example:

python movie_recommender.py --user "Chris Duncan"

If everything goes fine, you should see the following results:

```
Movie recommendations for Chris Duncan:
1. Vertigo
2. Goodfellas
3. Scarface
4. Roman Holiday
```

16. Save and submit your mini-project program code with results on: Chris Duncan, David Smith and Bill Duffy.

6.7 Summary

In this workshop, we learned how to extract **K-nearest neighbors (KNN)** for a given data point from a given dataset.

We then used this concept to build the **K-nearest neighbors (KNN)** classifier.

We discussed how to compute similarity scores such as the **Euclidean** and **Pearson scores**.

We learned how to use **collaborative filtering** to find similar users from a given dataset and used it to build a movie recommendation system.

Finally, we were able to test our model and run it against data points that the system had not previously seen.

In the next workshop, we will learn about **logic programming** and see how to build an inference engine that can solve real-world problems.

In []: ▶